

ARTIKATE STUDIO

503 Platinum Tower, Sohna Road, Gurugram, Haryana

Technical Assessment

Backend Developer

Python · Django · Systems Engineering

Time estimate 4 – 6 hours	Submission Git repository	Sections 4 required + 1 optional
-------------------------------------	-------------------------------------	--

Before You Begin

This assessment is designed to evaluate how you think through real engineering problems — not just whether you can write code that works. Every section requires written reasoning alongside your implementation. Submitting code without explanation will be treated as an incomplete submission.

What to submit

- A Git repository (GitHub or GitLab) — public or shared with the contact who sent you this document
- A `README.md` at the root with clear instructions to set up and run your code locally. We must be able to run everything in under 5 minutes.
- A `DESIGN.md` documenting your architectural decisions for Section 2 (Job Queue)
- Written answers clearly labelled per section — either as inline comments or in a separate `ANSWERS.md`
- All tests must pass from a clean environment. Code that does not run scores zero for that section.

Expectations

- All four numbered sections are required. Section 5 (screen recording) is optional.
- You may use any references, documentation, or tools you normally use at work.
- We value clear reasoning over clever code. A simple solution with a well-explained trade-off beats a complex one with no explanation.
- Do not over-engineer. Readable code that solves the problem is the goal.
- Your written explanations must be specific — name the Django component, the Redis command, the ORM behaviour. Generic statements do not demonstrate understanding.

01 Diagnose a Broken System

Scenario

A Django API endpoint `/api/orders/summary/` is used by a mobile app dashboard. Under normal load it responds in ~80ms. After a routine deployment it started timing out at 30 seconds or more for users with more than 200 orders. No code change was made to that view.

Your task

Investigate the problem, identify the root cause, and fix it. Document your thinking throughout.

Deliverables

- Write an **incident investigation log** — what you checked first, second, and why. No code needed here. We want to see your structured thought process.
- Identify the root cause category (N+1 query, missing index, ORM misconfiguration, serializer overhead, cache invalidation, etc.) and **justify your choice**.
- Write a realistic Django view or queryset that demonstrates the problem. You construct this yourself.
- Fix the code. In your explanation, describe **why the fix works at the database and ORM level** — not just that it does.
- Integrate `django-silk` or `django-debug-toolbar` and include evidence of the query count before and after the fix.

What we look for

A clear, ordered investigation narrative — not a list of guesses
Accurate identification of the ORM behaviour causing the regression
An explanation that references specific Django/database mechanisms
Profiler output showing measurable improvement

02 Design a Rate-Limited Async Job Queue

Scenario

You need to send transactional emails for a platform (order confirmations, OTP, alerts). The third-party email provider limits you to 200 emails per minute. The system receives bursts of 2,000 requests in under 10 seconds during flash sales. Build a queue that respects the rate limit, retries on failure, and does not lose jobs if the worker crashes mid-run.

Your task

Design, implement, and document the job queue system. Architecture decisions are as important as working code.

Deliverables

- Write a `DESIGN.md` covering your architecture choice — **Celery + Redis vs. Django Q vs. custom implementation** — with explicit trade-offs for each option. State what you chose and why.
- Implement the queue using **Celery with a Redis backend**: task definition, exponential backoff retry logic, and dead-letter handling for permanently failed jobs.
- Implement the rate limiter using a **Redis-based token-bucket or sliding-window approach**. Do not use `time.sleep()`. Your implementation must use Redis atomic operations.
- Write a test that submits 500 jobs and asserts: no job is lost, the rate limit is never exceeded, and at least one intentional failure is retried correctly.
- In your written answers, explain: **what happens to in-flight tasks if the Celery worker process is SIGKILL'd?** How does your implementation handle this?

Rate limiter — expected approach

Your rate limiter should use one of these patterns. Explain your choice in `DESIGN.md`.

```
# Option A: Token bucket    - Redis DECR + TTL
# Option B: Sliding window - Redis sorted set + ZREMRANGEBYSCORE
# Option C: Fixed window   - Redis INCR + EXPIRE
#
# For whichever you choose, explain:
#   1. Why this approach over the others
#   2. How you guarantee atomicity (Lua script, MULTI/EXEC, or pipeline)
#   3. What happens under Redis failure - does the app fail open or closed?
```

What we look for

A `DESIGN.md` that reasons through trade-offs, not just lists options
 A rate limiter you built — not a third-party library wrapping Redis
 A written SIGKILL answer that names specific Celery configuration (`ack_late`, etc.)
 A test that actually runs and asserts meaningful behaviour

03 Multi-Tenant Data Isolation

Scenario

You are building a SaaS platform where each client (tenant) must only ever see their own data. Isolation must be enforced at the ORM level — a single missed `.filter()` call must never expose another tenant's data. The enforcement must be automatic, not dependent on developers remembering to add a filter on every query.

Your task

Implement automatic tenant scoping at the ORM layer and explain its failure modes.

Starting scaffold

```
class TenantManager(models.Manager):
    # Implement this so that Order.objects.all()
    # automatically applies .filter(tenant=current_tenant)
    # even when the caller has no knowledge of tenant context
    pass

class Order(models.Model):
    tenant = models.ForeignKey(Tenant, on_delete=models.CASCADE)
    objects = TenantManager()
```

Deliverables

- Implement a custom Django `Manager` that automatically scopes all queriesets to the current tenant, using thread-local storage or request middleware.
- Write middleware that extracts the tenant from a subdomain or JWT header and sets it for the full request lifecycle.
- Write tests proving: (a) tenant A **cannot** access tenant B's data through any ORM call, and (b) calling `.objects.all()` does not bypass scoping.
- In your written answers, explain: **what are the failure modes of thread-local based tenant scoping in async Django views?** What would you change to make it safe for async, and why?

What we look for

A `Manager` implementation that cannot be accidentally bypassed

Middleware that correctly binds and cleans up tenant context per request

Tests that prove the negative — that the bypass fails, not just that it works

A written async answer that names Python's contextvars and explains why thread-locals fail

04 Written Architecture Review

No code required for this section. Answer any **two** of the three questions below in 200–350 words each. Answers must be specific. Naming a Django component, ORM method, or configuration setting earns credit. Generic statements do not.

Question A — Django Admin Performance

A client reports their Django admin page loads slowly when there are 500,000+ records in a table. They have already added a database index on the primary key.

Identify three other root causes you would investigate and explain specifically how you would fix each. For each, name the exact Django mechanism, model method, or configuration you would change.

Question B — Pagination Trade-offs

You are adding pagination to an existing API endpoint that currently returns 10,000 records. Two engineers propose: (a) offset-based pagination, and (b) cursor-based pagination.

Explain the real-world consequences of each approach at scale. Consider: mobile app infinite scroll, data mutation during pagination, and database scan behaviour. When would you choose one over the other?

Question C — File Upload Security

Your Django application handles user file uploads. A security audit flags the endpoint as high-risk.

List five distinct attack vectors specific to file uploads and explain how you would mitigate each one at the Django application layer. Do not describe CDN or WAF mitigations.

What we look for

Specific mechanism names — not 'optimise queries' but 'add `list_select_related` to `ModelAdmin`'

Honest acknowledgement of trade-offs — what does your choice sacrifice?

Answers grounded in how Django and the underlying database actually behave

★ Optional: Live System Recording

This section is entirely optional. It carries bonus marks and requires no additional coding beyond Section 2.

Record a 5–10 minute screen recording (Loom or equivalent) where you:

- Run your Section 2 queue implementation from a fresh terminal
- Submit 100+ jobs and show the Redis queue state in real time
- Show the rate limiter throttling — demonstrate it never exceeds 200 emails/minute
- Show at least one failure being retried with backoff
- Narrate what you are seeing and why it behaves that way

Submission

Include the Loom link (or equivalent) in your README.md.

The recording does not need to be edited or polished — a single raw take is fine.

What We Evaluate

We review every submission across four dimensions. There are no trick questions and no hidden pass/fail lines — we are looking for evidence of genuine engineering thinking.

Dimension	What this means in practice
Code quality	Is the code structured, readable, and correct? Does it handle edge cases? Would another engineer be able to modify it without asking you questions?
Written reasoning	Can you explain trade-offs, name specific mechanisms, and acknowledge what your approach does not handle well? Vague explanations score poorly regardless of code quality.
Test coverage	Do your tests actually run? Do they assert meaningful behaviour — not just that the happy path works, but that failures, retries, and constraints behave correctly?
Systems thinking	Do you understand why systems behave as they do, not just how to make them work? This shows in your incident log, your DESIGN.md, and your written answers.

We are not looking for

- Perfect code — clean, readable, and reasoning-backed beats clever and unexplained
- Exhaustive test coverage — a few precise tests that prove meaningful behaviour are better than many trivial ones
- A deployed application — local setup is sufficient
- Lengthy documentation — say what matters, skip what doesn't

Submission Checklist

Before sharing your repository, confirm each of the following:

- ☐ Git repository link shared with correct access permissions
- ☐ README.md present with clear setup and run instructions
- ☐ All four sections attempted and code files are present
- ☐ Written answers labelled per section in ANSWERS.md or inline
- ☐ DESIGN.md at root covering Section 2 architecture decisions
- ☐ All tests pass from a clean environment
- ☐ django-silk or django-debug-toolbar profiler evidence included for Section 1
- ☐ No .env files, secret keys, or credentials committed to the repository
- ☐ (Optional) Loom recording link in README.md

Questions about this assessment?

Reach out to the contact who shared this document. If something in the task is ambiguous, state your interpretation clearly in your ANSWERS.md and proceed — we will evaluate your reasoning, not just the outcome.